Projet R&D – Pullman Concept Room

Système d'authentification automatisée pour chambre intelligente

Rapport technique détaillé

Ethan Binisti – ESILV A5

Février 2026

Table des matières

1 Introduction **3**
1.1 Contexte du projet . . . 3
1.2 Objectifs principaux . . . 3

2 Architecture générale du système **3**
2.1 Vue d'ensemble . . . 3
2.2 Schéma de communication . . . 3
2.3 Technologies utilisées . . . 4
2.3.1 Backend (GRMS API) . . . 4
2.3.2 Frontend Web (Dashboard) . . . 4
2.3.3 Room Core (Simulation) . . . 4
2.3.4 Application iOS . . . 4

3 Description détaillée des composants **5**
3.1 GRMS API (Backend) . . . 5
3.1.1 Rôle et responsabilités . . . 5
3.1.2 Structure des données . . . 5
3.1.3 Endpoints principaux . . . 6
3.1.4 Sécurité . . . 6
3.2 Dashboard Web (Interface d'administration) . . . 7
3.2.1 Rôle . . . 7
3.2.2 Structure de l'interface . . . 7
3.2.3 Captures d'écran du dashboard GRMS . . . 8
3.2.4 Fonctionnement technique . . . 8
3.3 Room Core (Cœur de chambre) . . . 8
3.3.1 Rôle . . . 8
3.3.2 Fonctionnalités . . . 8
3.3.3 Communication avec le GRMS . . . 9
3.4 Application iOS Client . . . 9
3.4.1 Rôle . . . 9
3.4.2 Architecture de l'application . . . 9
3.4.3 Captures d'écran de l'application iOS . . . 9
3.4.4 Fonctionnement détaillé . . . 9
3.4.5 Sécurité dans l'application . . . 10

4 Parcours utilisateur complet **10**
4.1 Scénario type : Check-in et ouverture de porte . . . 10
4.1.1 Étape 1 : Réservation (avant l'arrivée) . . . 10
4.1.2 Étape 2 : Check-in anticipé . . . 10
4.1.3 Étape 3 : Arrivée du client . . . 11
4.1.4 Étape 4 : Ouverture de la porte . . . 11
4.1.5 Étape 5 : Check-out . . . 11
4.2 Scénario d'intrusion . . . 11

5 Installation et déploiement **11**
5.1 Prérequis . . . 11
5.2 Installation du backend GRMS . . . 12
5.3 Installation du dashboard web . . . 12
5.4 Installation du Room Core . . . 12
5.5 Configuration de l'application iOS . . . 12

6 Points techniques importants **13**
6.1 Gestion de la mémoire . . . 13
6.2 Gestion des dates . . . 13
6.3 Génération des tokens . . . 13
6.4 Communication HTTP . . . 13

7 Limitations et perspectives **14**
7.1 Limitations actuelles . . . 14
7.2 Perspectives d'amélioration . . . 14
7.2.1 Court terme . . . 14
7.2.2 Moyen terme . . . 14
7.2.3 Long terme . . . 14

8 Conclusion **15**

1 Introduction

1.1 Contexte du projet

Accor, leader mondial de l'hôtellerie, souhaite moderniser l'expérience client dans ses hôtels Pullman en développant un système d'authentification sans contact pour les chambres. L'objectif est de permettre aux clients d'ouvrir leur chambre directement depuis leur smartphone, sans avoir besoin de clé physique ou de carte magnétique.

Ce projet de recherche et développement vise à créer une **maquette fonctionnelle** complète d'un système de gestion de chambres intelligentes, depuis le check-in jusqu'à l'ouverture de porte, en passant par la gestion des accès et la sécurité.

1.2 Objectifs principaux

Les objectifs de ce projet sont multiples :

- **Sécuriser l'authentification** : Créer un système de tokens (clés numériques) sécurisés et temporaires pour chaque client
- **Anonymiser les données** : Protéger les informations personnelles des clients tout en permettant l'authentification
- **Simplifier l'expérience client** : Réduire les actions physiques nécessaires (plus besoin de passer à la réception, plus de clé à perdre)
- **Gérer les accès** : Permettre la gestion centralisée de toutes les chambres depuis un tableau de bord (GRMS)
- **Traçabilité** : Enregistrer tous les événements (check-in, tentatives d'ouverture, intrusions) dans des logs
- **Préparer l'intégration BLE** : Créer une architecture prête pour une future intégration Bluetooth Low Energy (BLE) avec iOS

2 Architecture générale du système

2.1 Vue d'ensemble

Le système est composé de **quatre composants principaux** qui communiquent entre eux :

1. **GRMS API** : Le serveur backend qui gère toute la logique métier
2. **Dashboard Web** : L'interface d'administration pour le personnel de l'hôtel
3. **Room Core** : Le système embarqué dans chaque chambre (simulé par un serveur Node.js)
4. **Application iOS Client** : L'application mobile pour les clients

2.2 Schéma de communication

Le flux de communication suit ce schéma (représenté avec des flèches simples pour rester lisible dans le PDF) :

Client iOS (Smartphone)

↓ JWT Auth & Token Request

GRMS API (Backend) ↔ **Dashboard Web (Admin)**

↓ Token Verification

Room Core (Raspberry Pi)

2.3 Technologies utilisées

2.3.1 Backend (GRMS API)

- **Node.js** : Environnement d'exécution JavaScript côté serveur
- **Express** : Framework web minimaliste pour créer des APIs REST
- **JSON Web Token (JWT)** : Système d'authentification sécurisé
- **CORS** : Mécanisme pour autoriser les requêtes cross-origin

Pourquoi ces technologies ?

- Node.js permet d'utiliser JavaScript aussi bien côté client que serveur, facilitant le développement
- Express est simple, léger et parfait pour créer rapidement une API REST
- JWT permet une authentification stateless (sans session serveur), idéale pour les applications mobiles

2.3.2 Frontend Web (Dashboard)

- **React** : Bibliothèque JavaScript pour créer des interfaces utilisateur
- **Vite** : Outil de build ultra-rapide pour le développement
- **JavaScript (ES6+)** : Langage de programmation moderne

Pourquoi React ?

- React permet de créer des interfaces réactives qui se mettent à jour automatiquement
- Composants réutilisables facilitent la maintenance du code
- Grande communauté et nombreux outils disponibles

2.3.3 Room Core (Simulation)

- **Node.js** : Même environnement que le backend pour la cohérence
- **Express** : Pour créer un mini-serveur HTTP dans chaque chambre
- **Axios** : Bibliothèque pour faire des requêtes HTTP vers le GRMS

2.3.4 Application iOS

- **Swift** : Langage de programmation officiel d'Apple pour iOS
- **SwiftUI** : Framework moderne d'Apple pour créer des interfaces
- **CoreBluetooth** : Framework pour la communication BLE (prévu pour la suite)
- **URLSession** : API native pour les requêtes HTTP

Pourquoi Swift et SwiftUI ?

- Swift est le langage natif recommandé par Apple pour iOS
- SwiftUI permet de créer des interfaces modernes avec moins de code
- Performance optimale sur les appareils Apple

3 Description détaillée des composants

3.1 GRMS API (Backend)

3.1.1 Rôle et responsabilités

Le GRMS API est le **cœur du système**. Il centralise toute la logique métier :

- Gestion des clients et de leurs données
- Gestion des réservations
- Génération et validation des tokens d'accès
- Gestion de l'état des chambres (libre, occupée, verrouillée)
- Enregistrement de tous les événements dans des logs
- Authentification JWT pour sécuriser l'accès à l'application mobile

3.1.2 Structure des données

Le système utilise un **stockage en mémoire** (pour la maquette). En production, ces données seraient stockées dans une base de données.

Clients Chaque client est représenté par :

- `id` : Identifiant unique
- `name` : Nom du client
- `email` : Adresse email (utilisée pour l'authentification)
- `password` : Mot de passe hashé (pour l'authentification JWT)
- `phone` : Numéro de téléphone (optionnel)
- `status` : Statut du client

Chambres Chaque chambre contient :

- `id` : Numéro de la chambre (ex : 101, 102)
- `status` : État actuel (`free`, `occupied`, `locked`)
- `lockedUntil` : Date jusqu'à laquelle la chambre est verrouillée (en cas d'intrusion)

Tokens Les tokens sont des clés numériques temporaires :

- `value` : Valeur aléatoire unique du token (ex : "101-1771317699037-ycx1pqo0c9")
- `clientId` : ID du client propriétaire
- `roomId` : Numéro de la chambre concernée
- `valid` : Booléen indiquant si le token est toujours valide
- `validFrom` : Date de début de validité (timestamp)
- `validTo` : Date de fin de validité (timestamp)

Réservations Les réservations lient clients et chambres :

- `id` : Identifiant unique
- `clientId` : ID du client
- `roomId` : Numéro de la chambre
- `startDate` : Date de début du séjour
- `endDate` : Date de fin du séjour
- `status` : État (`pending`, `in_progress`, `completed`)

Logs Chaque événement est enregistré :

- `id` : Identifiant unique
- `type` : Type d'événement (`CHECKIN`, `AUTH_ATTEMPT`, `CHECKOUT`, etc.)
- `roomId` : Chambre concernée
- `timestamp` : Date et heure de l'événement
- `payload` : Données supplémentaires (JSON)

3.1.3 Endpoints principaux

Authentification

- `POST /auth/login` : Connexion d'un client avec email/mot de passe, retourne un JWT
- `GET /auth/me` : Vérifie le JWT et retourne les informations du client connecté
- `GET /auth/my-token` : Récupère le token actif pour le client connecté

Gestion des chambres

- `GET /rooms` : Liste toutes les chambres et leur état
- `POST /rooms/:id/unlock` : Déverrouille une chambre verrouillée

Check-in et Check-out

- `POST /checkin` : Génère un token, occupe la chambre, crée un log
- `POST /checkout` : Libère la chambre, invalide les tokens

Gestion des tokens

- `POST /tokens/verify` : Vérifie si un token est valide pour une chambre donnée

Clients

- `GET /clients` : Liste tous les clients
- `POST /clients` : Crée un nouveau client

Réservations

- `GET /reservations` : Liste toutes les réservations
- `POST /reservations` : Crée une nouvelle réservation
- `POST /reservations/:id/checkin` : Effectue un check-in depuis une réservation

Logs

- `GET /logs` : Récupère les logs récents
- `POST /room-events` : Enregistre un événement depuis le room-core

3.1.4 Sécurité

Authentification JWT Le système utilise JSON Web Tokens pour authentifier les clients sur l'application mobile. Le processus est le suivant :

1. Le client se connecte avec son email et mot de passe via `POST /auth/login`
2. Le serveur vérifie les identifiants
3. Si valides, le serveur génère un JWT contenant l'ID du client et une date d'expiration

4. Le JWT est renvoyé au client et stocké dans l'application mobile
5. Pour chaque requête suivante, le client envoie le JWT dans l'en-tête `Authorization: Bearer <token>`
6. Le serveur vérifie la signature et la validité du JWT avant de traiter la requête

Validation des tokens Avant d'autoriser l'ouverture d'une porte, le système vérifie :

— Que le token existe dans la base de données
— Que le token est associé à la bonne chambre
— Que le token est toujours valide (`valid = true`)
— Que la date actuelle est entre `validFrom` et `validTo`
— Que la chambre n'est pas verrouillée pour intrusion

Détection d'intrusion Si plusieurs tentatives d'authentification échouent consécutivement (3 échecs), le système :

— Verrouille la chambre (`status = "locked"`)
— Enregistre un événement `INTRUSION_LOCK` dans les logs
— Empêche toute nouvelle tentative pendant 2 minutes
— Notifie le personnel via le dashboard

3.2 Dashboard Web (Interface d'administration)

3.2.1 Rôle

Le dashboard web est l'**interface d'administration** utilisée par le personnel de l'hôtel pour :

— Visualiser l'état de toutes les chambres en temps réel
— Effectuer les check-in et check-out
— Gérer les clients et leurs réservations
— Consulter les logs d'activité
— Gérer les situations d'urgence (déverrouillage de chambres)

3.2.2 Structure de l'interface

L'interface est organisée en plusieurs sections :

Section Check-in Permet de :

— Sélectionner un client dans une liste déroulante
— Choisir une chambre disponible
— Définir la durée de validité du token (en minutes)
— Générer le token et effectuer le check-in
— Libérer la chambre (check-out)

Section Chambres Affiche :

— Toutes les chambres avec leur statut visuel (couleur)
— Le nombre de chambres libres, occupées, verrouillées
— Un bouton pour simuler une tentative d'accès (pour les tests)

Section Clients & Réservations Permet de :

- Créer de nouveaux clients (fichier client)
- Voir la liste des clients existants
- Créer des réservations (client + chambre + dates)
- Effectuer un check-in anticipé depuis une réservation

Section Logs Affiche en temps réel :

- Tous les événements récents (check-in, tentatives d'accès, intrusions, etc.)
- Le type d'événement, la chambre concernée, le succès/échec, l'heure
- Filtrage possible par type d'événement

3.2.3 Captures d'écran du dashboard GRMS

Pour illustrer concrètement l'outil d'administration, les figures suivantes montrent le dashboard en situation réelle :

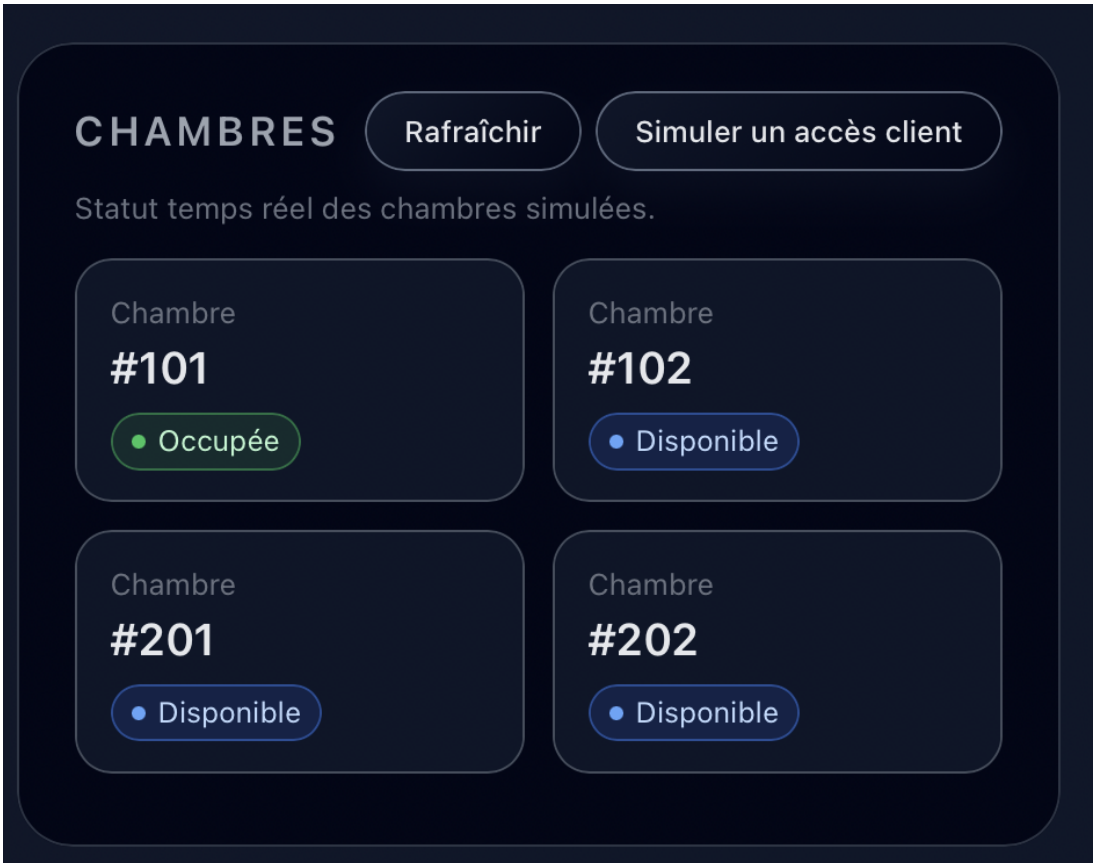

FIGURE 1 – Vue "Chambres" du dashboard GRMS : affichage en temps réel du statut des chambres (disponible, occupée, etc.).

3.2.4 Fonctionnement technique

Le dashboard est une **application React** qui :

- Se connecte au GRMS API via des requêtes HTTP
- Met à jour automatiquement l'interface quand les données changent
- Utilise des appels `fetch()` pour communiquer avec le backend
- Affiche les données en temps réel grâce au rafraîchissement automatique

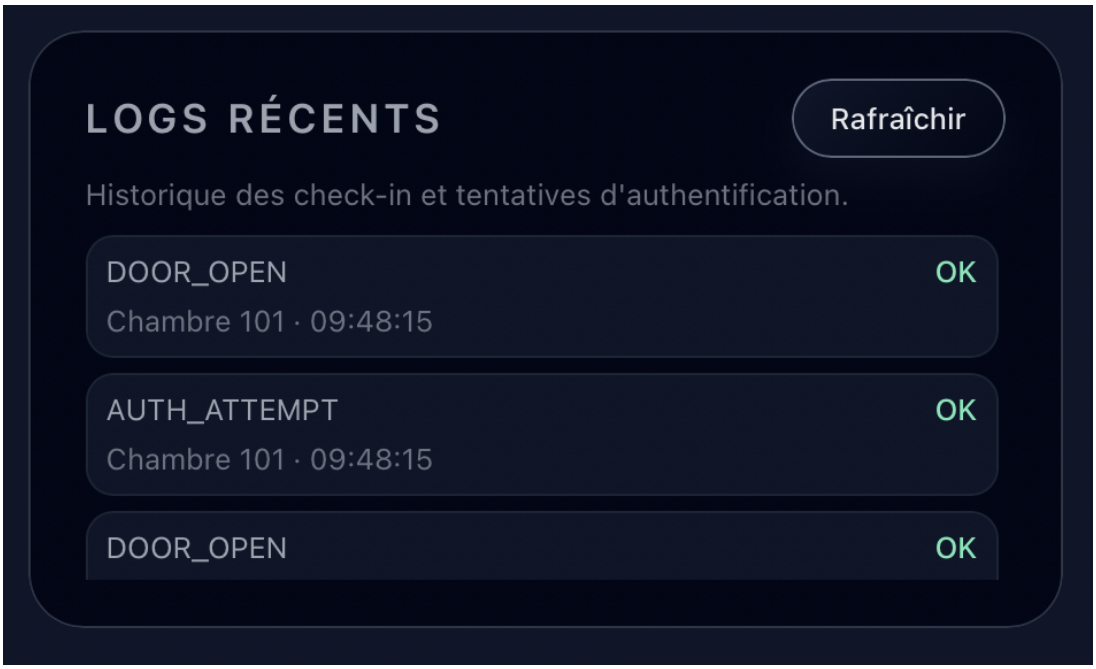

FIGURE 2 – Vue "Logs récents" du dashboard GRMS : historique des ouvertures de porte et tentatives d'authentification.

3.3 Room Core (Cœur de chambre)

3.3.1 Rôle

Le Room Core simule le système embarqué qui serait installé dans chaque chambre (typiquement un Raspberry Pi). Il représente le **point d'entrée physique** de la chambre.

3.3.2 Fonctionnalités

Authentification locale

- Reçoit les tentatives d'authentification depuis le smartphone du client
- Vérifie le token avec le GRMS API
- Gère un compteur local d'échecs consécutifs
- Peut se verrouiller localement en cas de trop d'échecs

Simulation d'ouverture de porte

- Simule l'ouverture de la porte électronique
- Enregistre l'événement dans le GRMS
- Gère l'état de la porte (ouverte, fermée, verrouillée)

Sécurité locale

- Maintient un verrouillage local temporaire après 3 échecs
- Communique avec le GRMS pour signaler les intrusions
- Peut être déverrouillé à distance depuis le dashboard

3.3.3 Communication avec le GRMS

Le Room Core communique avec le GRMS via HTTP :

- `POST /tokens/verify` : Vérifie un token avec le GRMS
- `POST /room-events` : Envoie des événements (ouverture de porte, verrouillage) au GRMS

3.4 Application iOS Client

3.4.1 Rôle

L'application iOS est l'**interface client** qui permet aux clients de :

- Se connecter avec leurs identifiants
- Récupérer automatiquement leur clé numérique après le check-in
- Ouvrir leur chambre en approchant leur smartphone du lecteur

3.4.2 Architecture de l'application

L'application est structurée selon le pattern **MVVM** (Model-View-ViewModel) :

Modèles (Models)

- `User` : Structure représentant un utilisateur (id, name, email)
- `TokenResponse` : Structure pour la réponse contenant le token

Vues (Views)

- `ContentView` : Interface principale avec deux vues :
  - Vue de connexion (login)
  - Vue principale (affichage de la clé, bouton d'ouverture)

ViewModels

- `ClientViewModel` : Gère l'état de l'application et la logique métier

Services

- `AuthService` : Gère l'authentification JWT et le stockage local
- `ApiClient` : Gère toutes les communications HTTP avec le GRMS et le Room Core

3.4.3 Captures d'écran de l'application iOS

Les figures suivantes présentent l'écran principal de l'application client iOS ainsi qu'un rappel visuel du parcours clé numérique :

3.4.4 Fonctionnement détaillé

Connexion

1. L'utilisateur entre son email et mot de passe
2. L'application envoie une requête `POST /auth/login` au GRMS
3. Le GRMS vérifie les identifiants et retourne un JWT
4. L'application stocke le JWT dans `UserDefaults` (stockage local iOS)
5. L'application récupère automatiquement le token actif via `GET /auth/my-token`

Récupération du token

1. Après connexion, l'application appelle automatiquement `GET /auth/my-token`
2. Le GRMS vérifie le JWT et retourne le token actif pour ce client
3. Le token est stocké dans le ViewModel (mais **non affiché** à l'utilisateur pour la sécurité)
4. La date de validité est affichée à l'utilisateur

Ouverture de porte

1. L'utilisateur appuie sur "Approcher du lecteur"
2. L'application envoie le token au Room Core via `POST /auth`
3. Le Room Core vérifie le token avec le GRMS
4. Si valide, le Room Core simule l'ouverture de la porte
5. L'application affiche un message de succès

3.4.5 Sécurité dans l'application

Stockage sécurisé

- Le JWT est stocké dans `UserDefaults`, qui est chiffré par iOS
- Le token de chambre n'est **pas affiché** à l'utilisateur (masqué pour la sécurité)
- Seul un indicateur visuel (cadenas) indique si une clé est active

Authentification

- Toutes les requêtes vers le GRMS incluent le JWT dans l'en-tête `Authorization`
- Le JWT expire après un certain temps (configuré dans le GRMS)
- En cas d'expiration, l'utilisateur doit se reconnecter

4 Parcours utilisateur complet

4.1 Scénario type : Check-in et ouverture de porte

Voici le parcours complet d'un client depuis son arrivée jusqu'à l'ouverture de sa chambre :

4.1.1 Étape 1 : Réservation (avant l'arrivée)

1. Le personnel de l'hôtel crée une réservation dans le dashboard
2. La réservation lie un client à une chambre pour des dates données
3. Le statut de la réservation est `pending` (en attente)

4.1.2 Étape 2 : Check-in anticipé

1. Le personnel peut effectuer un check-in anticipé depuis le dashboard
2. Le système génère un token valide pour la période du séjour
3. La chambre passe en statut `occupied`
4. Un événement `CHECKIN_FROM_RESERVATION` est enregistré dans les logs

4.1.3 Étape 3 : Arrivée du client

1. Le client ouvre l'application iOS sur son smartphone
2. Il se connecte avec son email et mot de passe (créés lors de la réservation)
3. L'application authentifie le client et récupère un JWT
4. L'application récupère automatiquement le token actif pour sa chambre
5. L'application affiche un indicateur visuel (cadenas vert) indiquant qu'une clé est active
6. La date de validité est affichée

4.1.4 Étape 4 : Ouverture de la porte

1. Le client se rend devant sa chambre
2. Il ouvre l'application et appuie sur "Approcher du lecteur"
3. L'application envoie le token au Room Core de la chambre
4. Le Room Core vérifie le token avec le GRMS
5. Si valide, le Room Core simule l'ouverture de la porte
6. Un événement `DOOR_OPEN` est enregistré dans les logs
7. L'application affiche "Accès autorisé – la porte s'ouvre !"

4.1.5 Étape 5 : Check-out

1. À la fin du séjour, le personnel effectue le check-out depuis le dashboard
2. Le système invalide tous les tokens de la chambre
3. La chambre repasse en statut `free`
4. Un événement `CHECKOUT` est enregistré
5. Le client ne peut plus ouvrir la chambre (son token est invalidé)

4.2 Scénario d'intrusion

Si quelqu'un tente d'ouvrir une chambre sans autorisation :

1. Une tentative d'authentification avec un token invalide est effectuée
2. Le Room Core vérifie avec le GRMS et reçoit une réponse négative
3. Le compteur local d'échecs est incrémenté
4. Après 3 échecs consécutifs, le Room Core se verrouille localement
5. La chambre passe en statut `locked` dans le GRMS
6. Un événement `INTRUSION_LOCK` est enregistré
7. Le personnel est alerté via le dashboard
8. La chambre reste verrouillée pendant 2 minutes minimum
9. Le personnel peut déverrouiller manuellement depuis le dashboard si nécessaire

5 Installation et déploiement

5.1 Prérequis

Pour faire fonctionner le système, il faut :

— **Node.js** version 18 ou supérieure

— **npm** (gestionnaire de paquets Node.js, installé avec Node.js)
— **macOS** pour développer l'application iOS (Xcode requis)
— Un navigateur web moderne (Chrome, Firefox, Safari, Edge)
— (Optionnel) **TeX Live** pour compiler le rapport LaTeX

5.2 Installation du backend GRMS

1. Ouvrir un terminal et naviguer vers le dossier `grms-api`
2. Installer les dépendances : `npm install`
3. Lancer le serveur : `node index.js`
4. Le serveur démarre sur `http://localhost:4000`

Dépendances installées :

— `express` : Framework web
— `cors` : Gestion des requêtes cross-origin
— `jsonwebtoken` : Génération et vérification de JWT

5.3 Installation du dashboard web

1. Ouvrir un nouveau terminal et naviguer vers `grms-web`
2. Installer les dépendances : `npm install`
3. Lancer le serveur de développement : `npm run dev - -host 0.0.0.0 -port 5174`
4. Le dashboard est accessible sur `http://localhost:5174`

Dépendances installées :

— `react` et `react-dom` : Bibliothèques React
— `vite` : Outil de build et serveur de développement

5.4 Installation du Room Core

1. Ouvrir un nouveau terminal et naviguer vers `room-core`
2. Installer les dépendances : `npm install`
3. Lancer le serveur : `node index.js`
4. Le Room Core démarre sur `http://localhost:5001`

Dépendances installées :

— `express` : Framework web
— `axios` : Client HTTP pour communiquer avec le GRMS
— `cors` : Gestion des requêtes cross-origin

5.5 Configuration de l'application iOS

1. Ouvrir le projet dans Xcode : `smart-room-ios/SmartRoomClient/SmartRoomClient.xcodeproj`
2. Vérifier que l'adresse IP du Mac est correcte dans `ApiClient.swift`
3. Compiler et lancer l'application sur un simulateur ou un appareil iOS

Configuration réseau : Pour que l'application iOS puisse communiquer avec le backend depuis un iPhone physique :

— Trouver l'adresse IP locale du Mac : `ipconfig getifaddr en0`
— Mettre à jour les URLs dans `ApiClient.swift` :
  — `grmsURL` : `http://<IP_MAC>:4000`
  — `roomCoreURL` : `http://<IP_MAC>:5001`
— Mettre à jour le fichier `.env` dans `grms-web` avec la même IP

6 Points techniques importants

6.1 Gestion de la mémoire

Pour cette maquette, toutes les données sont stockées **en mémoire** (dans des tableaux JavaScript). Cela signifie que :

- Les données sont perdues à chaque redémarrage du serveur
- C'est suffisant pour une démonstration
- En production, il faudrait utiliser une base de données (PostgreSQL, MongoDB, etc.)

6.2 Gestion des dates

Le système utilise des **timestamps Unix** (millisecondes depuis le 1er janvier 1970) pour stocker les dates. Cela permet :

- Des calculs simples (comparaisons, additions)
- Une conversion facile en dates lisibles pour l'affichage
- Une gestion des fuseaux horaires cohérente

6.3 Génération des tokens

Les tokens sont générés avec cette formule :

```
token = roomId + "-" + timestamp + "-" + randomString
```

Exemple : `101-1771317699037-ycx1pqo0c9`
Cela garantit :

- L'unicité du token
- La traçabilité (on peut voir la chambre et la date de génération)
- L'impossibilité de deviner un token valide

6.4 Communication HTTP

Toutes les communications entre les composants utilisent le protocole **HTTP** avec des requêtes **REST** :

- `GET` : Récupérer des données
- `POST` : Créer ou envoyer des données
- Les réponses sont au format **JSON** (JavaScript Object Notation)

Exemple de requête :

```
POST /auth/login
Content-Type: application/json

{
  "email": "client@example.com",
  "password": "motdepasse123"
}
```

Exemple de réponse :

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": 1,
    "name": "Jean Dupont",
    "email": "client@example.com"
  }
}
```

7 Limitations et perspectives

7.1 Limitations actuelles

Cette maquette présente plusieurs limitations volontaires :

- **Pas de persistance** : Les données sont perdues au redémarrage
- **Pas de BLE réel** : La communication Bluetooth n'est pas implémentée
- **Simulation du Room Core** : Le Room Core est un serveur HTTP, pas un vrai Raspberry Pi
- **Sécurité basique** : Les mots de passe ne sont pas hashés (pour simplifier la démo)
- **Pas de multi-utilisateurs** : Un seul client peut avoir un token actif par chambre à la fois

7.2 Perspectives d'amélioration

7.2.1 Court terme

- **Base de données** : Intégrer PostgreSQL ou MongoDB pour la persistance
- **Hashage des mots de passe** : Utiliser bcrypt pour sécuriser les mots de passe
- **Multi-smartphones** : Permettre à plusieurs appareils d'utiliser le même token
- **Notifications push** : Alerter le client quand sa clé est prête

7.2.2 Moyen terme

- **Intégration BLE** : Implémenter la communication Bluetooth Low Energy entre iOS et Room Core
- **Room Core physique** : Déployer sur de vrais Raspberry Pi avec lecteurs NFC/BLE
- **Application Android** : Développer une version Android de l'application client
- **API publique** : Créer une API documentée pour les intégrations tierces

7.2.3 Long terme

- **Intelligence artificielle** : Détecter les comportements suspects
- **Intégration avec les systèmes hôteliers** : Connexion avec les systèmes de réservation existants
- **Géolocalisation** : Ouvrir automatiquement la chambre quand le client arrive à proximité
- **Personnalisation** : Adapter l'environnement de la chambre (température, éclairage) selon les préférences

8 Conclusion

Ce projet démontre la faisabilité d'un système d'authentification sans contact pour chambres d'hôtel intelligentes. La maquette fonctionnelle couvre l'ensemble du parcours client, depuis la réservation jusqu'à l'ouverture de porte, avec une gestion centralisée via un GRMS.

Les technologies choisies (Node.js, React, Swift) sont modernes, performantes et permettent une évolution future vers une solution de production. L'architecture modulaire facilite l'ajout de nouvelles fonctionnalités et l'intégration de composants physiques (Raspberry Pi, lecteurs BLE).

Le système respecte les principes de sécurité (authentification JWT, tokens temporaires, détection d'intrusion) tout en offrant une expérience utilisateur fluide et moderne.

Cette maquette constitue une base solide pour un développement ultérieur vers une solution complète et déployable dans les hôtels Pullman d'Accor.

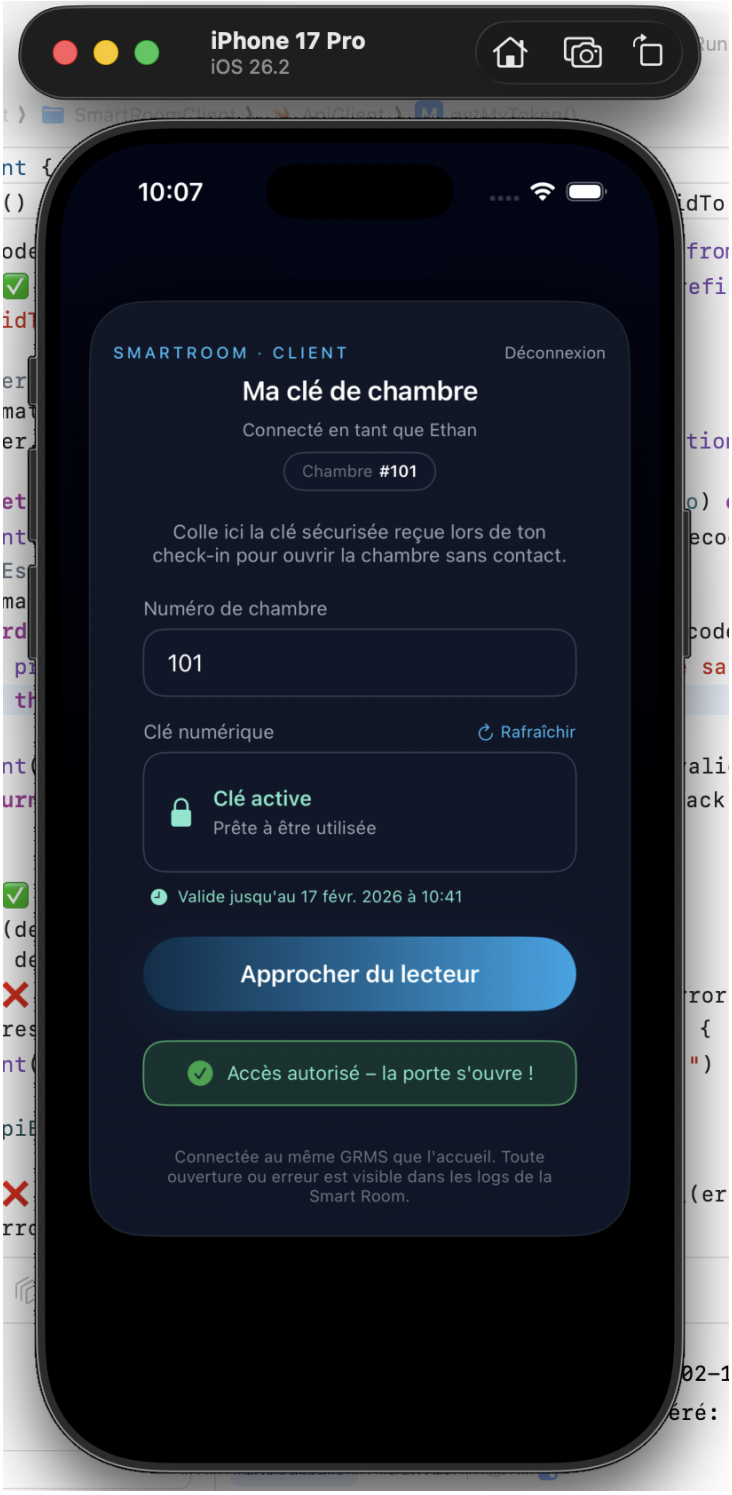

FIGURE 3 – Écran principal de l'application iOS : clé numérique active pour la chambre, date de validité et bouton "Approcher du lecteur".

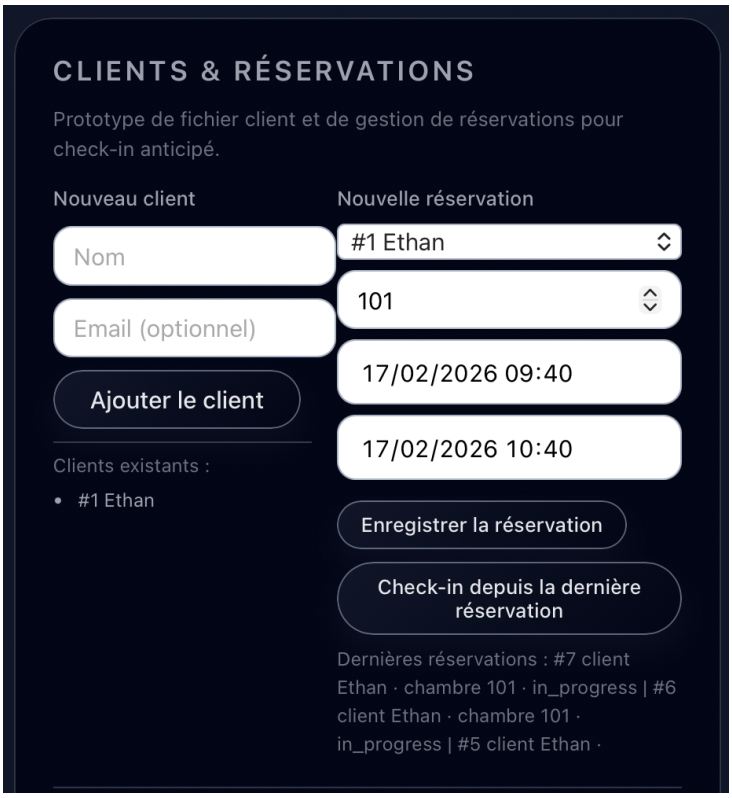

FIGURE 4 – Rappel visuel du lien entre l'application iOS et le GRMS : la clé numérique est issue d'un check-in côté accueil (clients & réservations).